

- 1) Quais as vantagens e desvantagens do método de análise sintática *LL(1)* em relação ao *LALR(1)*, cite dois geradores de compiladores que usam métodos de análise sintática *top-down* e dois geradores que usam métodos *bottom-up*.
- 2) Explique o que é escopo léxico. Cite duas linguagens de programação que possuem escopo léxico e duas que adotam escopo dinâmico.
- 3) Como é organizada a área de memória de dados durante a execução de um programa? Usando como exemplo a linguagem C, mostre que tipos de dados são alocados em cada uma das áreas.
- 4) Na linguagem C quais variáveis possuem escopo local e quais possuem escopo global?
- 5) No contexto de linguagens de programação e implementação de tradutores (compiladores/interpretadores), qual a diferença entre uma política de análise estática e política de análise dinâmica?
- 6) Qual a diferença entre a passagem de parâmetros por valor e passagem por referência, quando acontece cada uma delas na linguagem Java?
- 7) Defina uma gramática para expressões na notação pós-fixada envolvendo constantes inteiras (representadas pelo *token c*) e os operadores aritméticos: adição, subtração, multiplicação e divisão. Mostre a árvore sintática construída na derivação da expressão: **c c + c * c c - /**.
- 8) Como seria representada na notação infixa a expressão pós-fixada:

15 5 + 10 * 50 40 - /

- 9) Construa a tabelas sintática *SLR(1)* para a seguinte gramática:
P → T id (L)
T → int
| float
L → L , T id
| T id
- 10) Construa a tabela sintática *LL(1)* para a linguagem gerada pela gramática da questão anterior.
- 11) Para que serve a tabela de símbolos?
- 12) O que é uma árvore sintática abstrata?

- 13) No programa em linguagem C abaixo mostre um erro léxico, um erro sintático e um erro semântico. Explique cada um deles. No caso desses erros serem corrigidos o programa ainda apresentaria um comportamento indefinido (*undefined behavior*). Mostre onde e explique.

```
void main()
{
    float v[5], i;

    i = 1A;
    while (i <= 5
    {
        v[i] = i*10;
        i++;
    }
}
```

- 14) Mostre o código em linguagem *assembly* da JVM correspondente ao seguinte fragmento de código Java:

```
{
    int a, b, c;
    a = 2;
    c = 5;
    b = 0;
    if (a > 0 && a < c + 30)
    {
        b = (a + 10) * 2 + c;
    }
    System.out.println(b);
}
```